Experiment No. 6
To implement and compare different exploration strategies and analyze their impact on the agent's learning performance.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement and compare different exploration strategies in a reinforcement learning setting and analyze their impact on the agent's learning performance.

Objectives:

1. To understand the importance of **exploration strategies** in decision-making problems.
2. To implement common exploration techniques such as:
   - **Epsilon-Greedy**
   - **Upper Confidence Bound (UCB)**
3. To study the **exploration vs exploitation trade-off** in learning agents.

Theory

1. Exploration vs Exploitation

In reinforcement learning, an agent must choose between:

- **Exploration**: Trying new actions to gain more knowledge
- **Exploitation**: Using known information to maximize reward

Balancing these two is crucial for optimal learning.

2. Exploration Strategies

a) Epsilon-Greedy Strategy

- The agent selects a random action with probability **ε** (exploration).
- The agent selects the best-known action with probability **1 − ε** (exploitation).

Key Idea:
Ensures exploration but may not be optimal if ε is fixed.

b) Upper Confidence Bound (UCB)

- Selects actions based on both **estimated reward** and **uncertainty**.
- Gives preference to less-explored actions using a confidence term.

$$A_t = \arg\max_a \left(Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right)$$

Key Idea:
Balances exploration intelligently by considering confidence intervals.

3. Learning Performance Metrics

To analyze the effectiveness of strategies:

- **Cumulative Reward**: Total reward obtained over time
- **Regret**: Difference between actual reward and optimal reward
- **Convergence Rate**: Speed at which the agent learns the best action

4. Impact on Learning Performance

- Proper exploration helps the agent **discover optimal actions faster**.
- Poor exploration may lead to **suboptimal decisions**.
- Advanced methods like UCB typically provide **better long-term performance** compared to simple methods like epsilon-greedy.

Implementation:

```
import numpy as np

import matplotlib.pyplot as plt



# Step 1: Environment

n_arms = 5

true_probs = [0.2, 0.5, 0.75, 0.6, 0.4]  # true reward probabilities

n_steps = 1000



def pull_arm(arm):

    return 1 if np.random.rand() < true_probs[arm] else 0
```

```
optimal_reward = max(true_probs)


# -------------------------
# Epsilon-Greedy
# -------------------------
epsilon = 0.1
Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)

rewards_eps = []
regret_eps = []

for t in range(n_steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms)
    else:
        action = np.argmax(Q_eps)

    reward = pull_arm(action)
    N_eps[action] += 1
    Q_eps[action] += (reward - Q_eps[action]) / N_eps[action]

    rewards_eps.append(reward)
    regret_eps.append(optimal_reward - reward)
```

```
# -------------------------
# UCB Strategy
# -------------------------
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)

rewards_ucb = []
regret_ucb = []

c = 2

for t in range(n_steps):
    if t < n_arms:
        action = t
    else:
        ucb_values = Q_ucb + c * np.sqrt(np.log(t+1) / (N_ucb + 1e-5))
        action = np.argmax(ucb_values)

    reward = pull_arm(action)
    N_ucb[action] += 1
    Q_ucb[action] += (reward - Q_ucb[action]) / N_ucb[action]

    rewards_ucb.append(reward)
```

```
    regret_ucb.append(optimal_reward - reward)


# -------------------------
# Cumulative Reward
# -------------------------
cum_rewards_eps = np.cumsum(rewards_eps)
cum_rewards_ucb = np.cumsum(rewards_ucb)


# -------------------------
# Plot Results
# -------------------------
plt.figure(figsize=(10,5))


plt.subplot(1,2,1)
plt.plot(cum_rewards_eps, label="Epsilon-Greedy")
plt.plot(cum_rewards_ucb, label="UCB")
plt.title("Cumulative Reward")
plt.legend()


plt.subplot(1,2,2)
plt.plot(np.cumsum(regret_eps), label="Epsilon-Greedy")
plt.plot(np.cumsum(regret_ucb), label="UCB")
plt.title("Cumulative Regret")
plt.legend()
```

plt.show()

Conclusion:

1. Which Strategy Gives Optimal Solution?

- **UCB (Upper Confidence Bound)** generally provides better and more optimal solutions.
- It balances exploration and exploitation more effectively than Epsilon-Greedy.

---

2. Advantages and Disadvantages

Epsilon-Greedy

Advantages:

- Simple to implement
- Guarantees exploration

Disadvantages:

- Fixed ε may lead to too much or too little exploration
- Slower convergence
- Random exploration is inefficient

UCB

Advantages:

- Smart exploration using uncertainty
- Faster convergence
- Better long-term performance

Disadvantages:

- Slightly complex to implement
- Requires tuning of parameter *c*